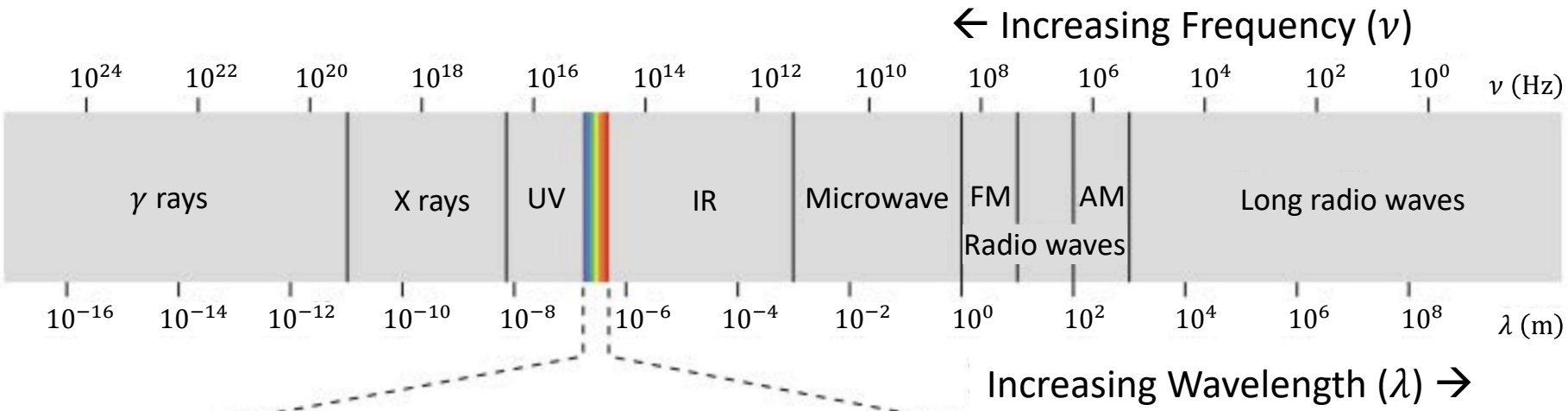
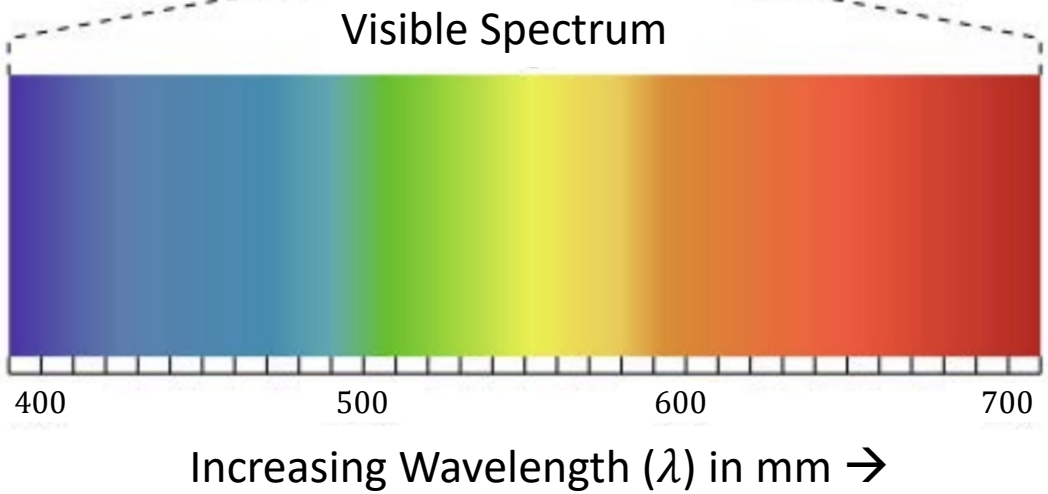
Lights and Shading

COMP557
Paul Kry

Light has Colour, Light has energy





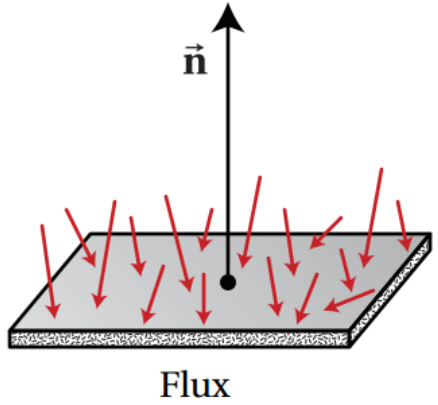
- Not all light is visible
- We can measure light by counting photons landing on a small area
- Care about visible photons, or those at a given wavelength

Measuring Light (Radiometry)

- Many physical processes convert energy into photons
  - Incandescent lightbulb turns heat into light (blackbody radiation)
  - Nuclear fusion in stars (sun) generates photons
  - Etc.
- Each photon carries a small amount of energy
- Want to record “how much energy”
- Energy of photons hitting an object is related to *brightness*
  - Film, eyes, CCD sensor
  - Need this information to make accurate images

Measuring Light

- Radiant Energy, Joules
  - Think of total number of photons that hit a sensor for a period of time, i.e., total energy, Joules
- Radiant flux, Φ, Watts
  - Think of hits per second on the sensor, or photons leaving a surface
  - The SI unit of *luminous* (i.e., visible) flux is the lumen (lm)

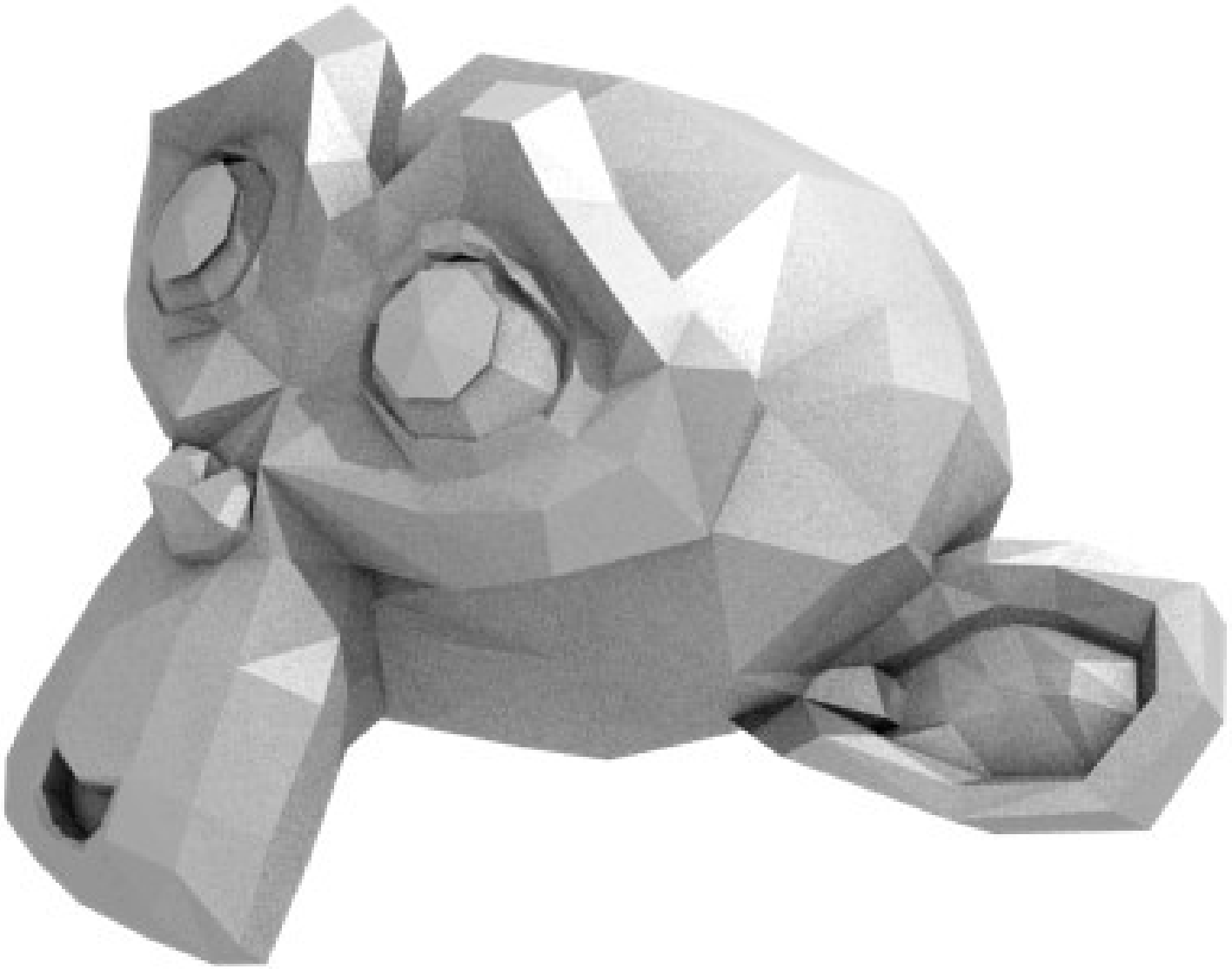


- Radiant flux *density*, is called *irradiance*
  - Hits per second per unit area
  - Luminous (visible) flux density is lux, lumen per meter squared

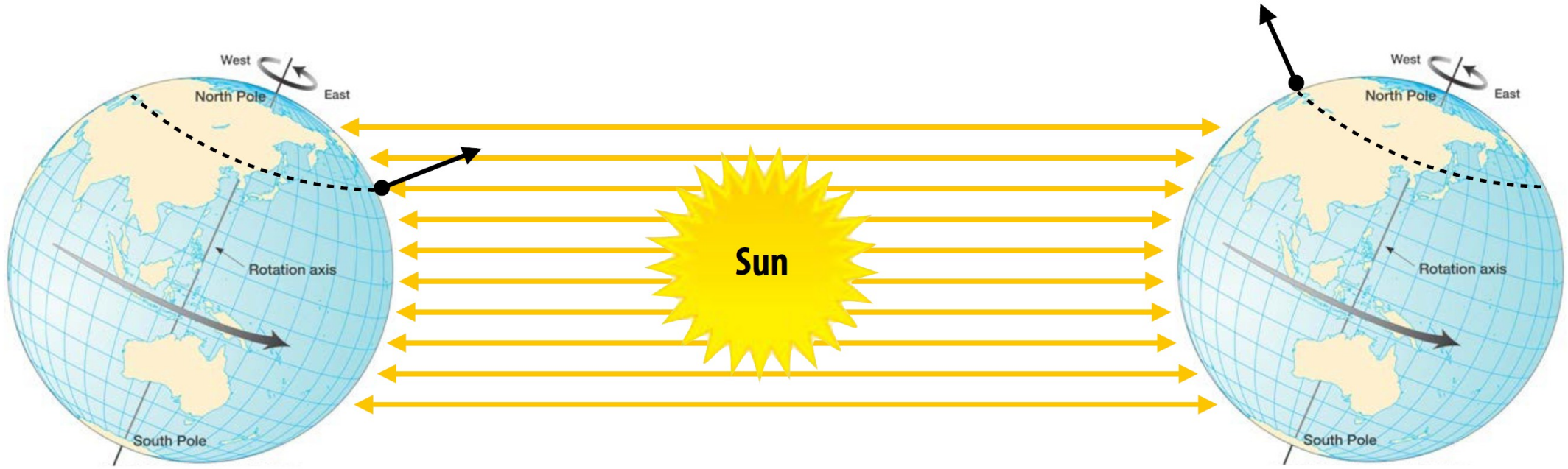
Radiant Flux and Radian Flux Density, lm vs lux

- One lux is one lumen per square meter
- Sunlight, 3.6×10^{28} lumens
  - About 100000 lux at earth
  - About 1000 Watts per meter square.
- Bright LED light bulb is ~ 800 lumens
  - Equivalent to 60 W incandescent

Why do some parts of a surface look lighter or darker?



Why do we have seasons?

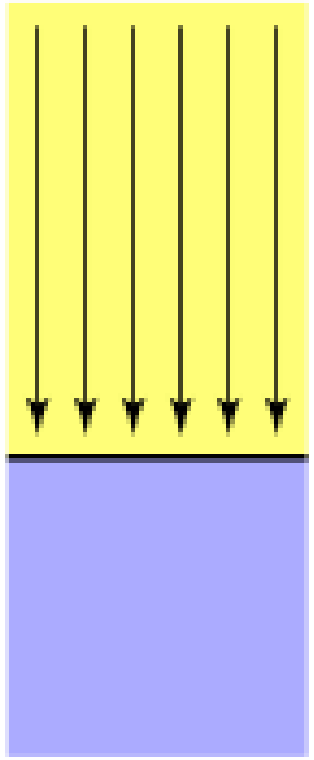


Summer (Northern Hemisphere)

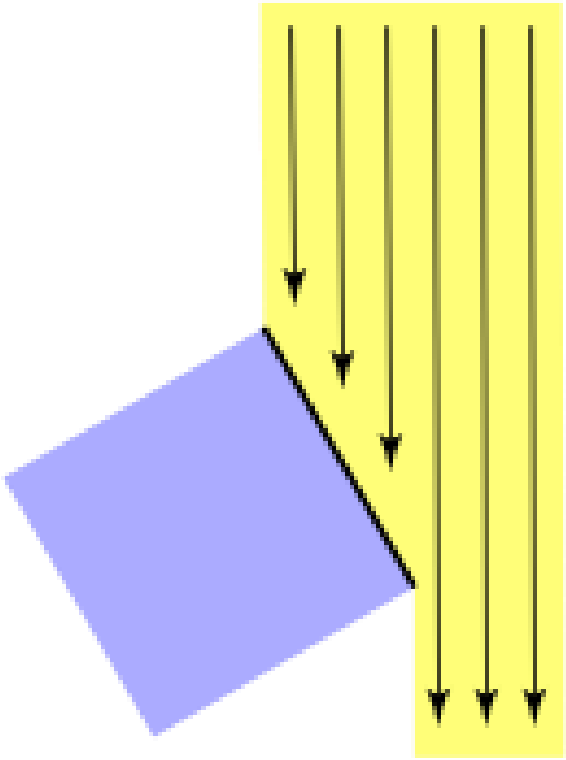
Winter (Northern Hemisphere)

Earth's axis of rotation $\sim 23.5°$ off axis of the plane of its orbit around the sun

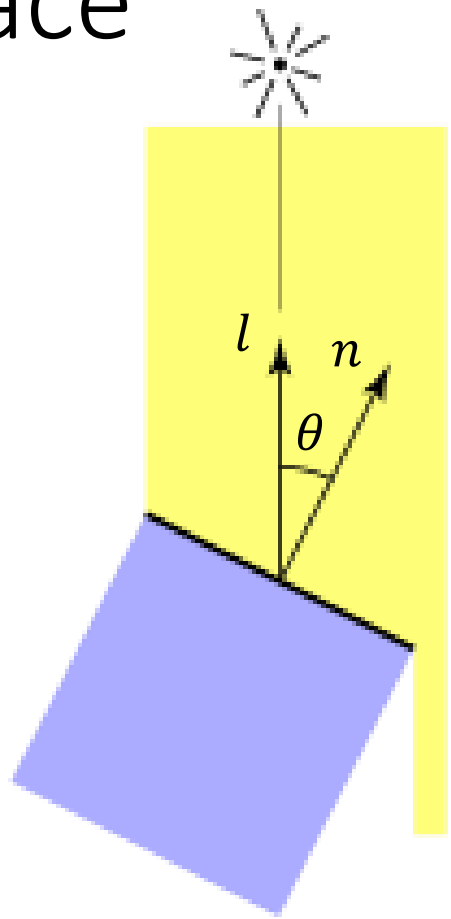
Light landing on a surface



Top face of cube receives a given amount of light



Top face of $60°$ rotated cube intercepts half the light



In general, light per unit area is proportional to $\cos\theta = l \cdot n$

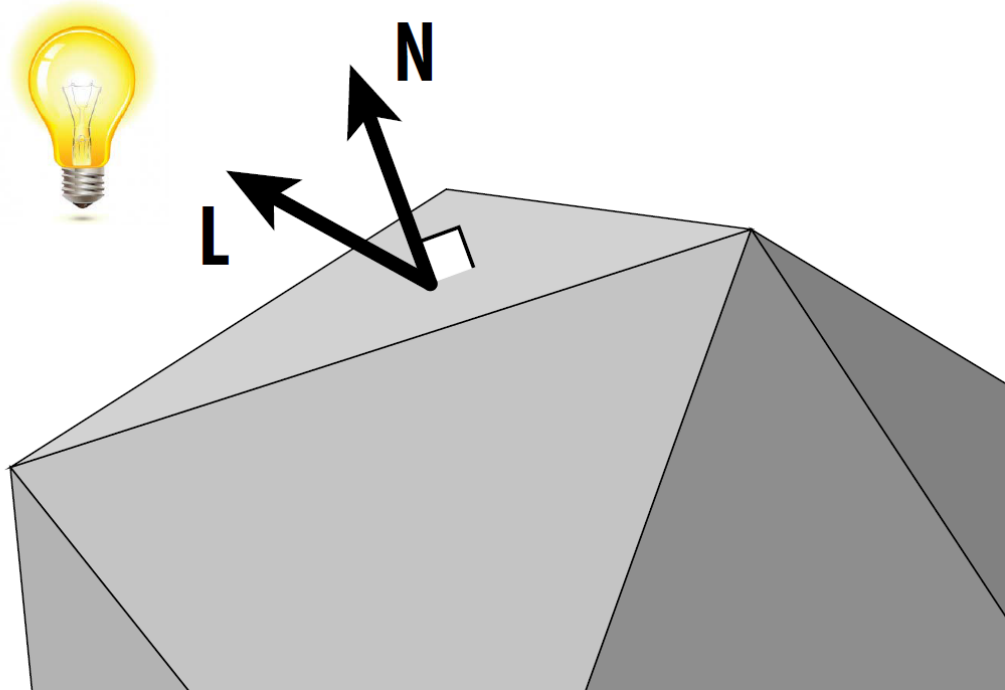
Light direction l
Surface normal n

“N-dot-L” shading

- Most basic way to shade a surface: take dot product of unit surface normal (N) and unit direction to light (L)

```
double surfaceColor( Vec3 N, Vec3 L ) {
        return dot( N, L );
}
```

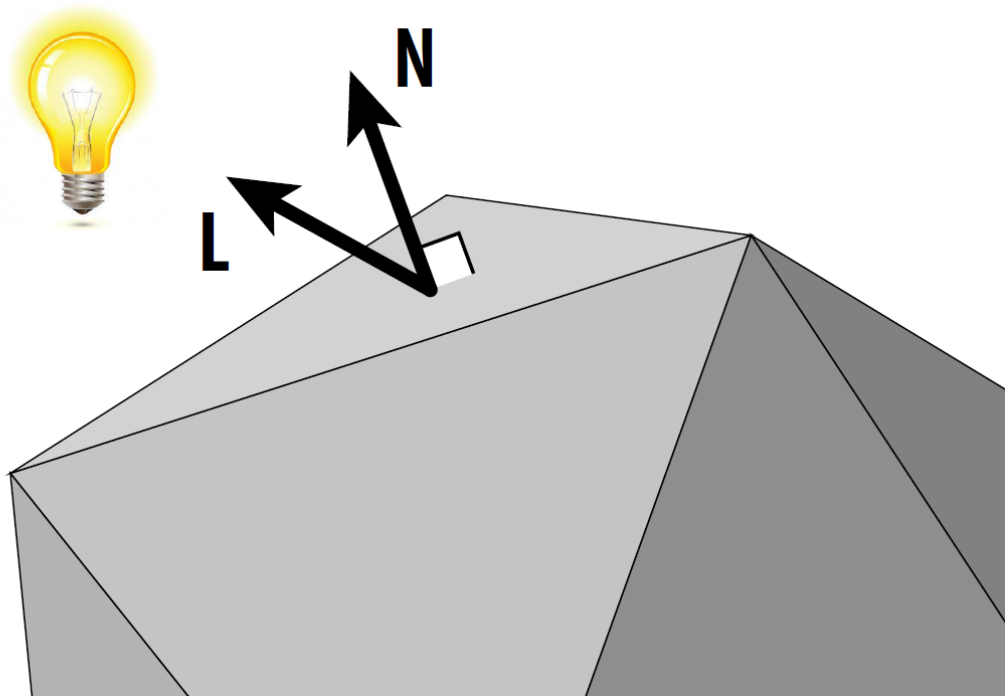
What is wrong with this code?



“N-dot-L” shading

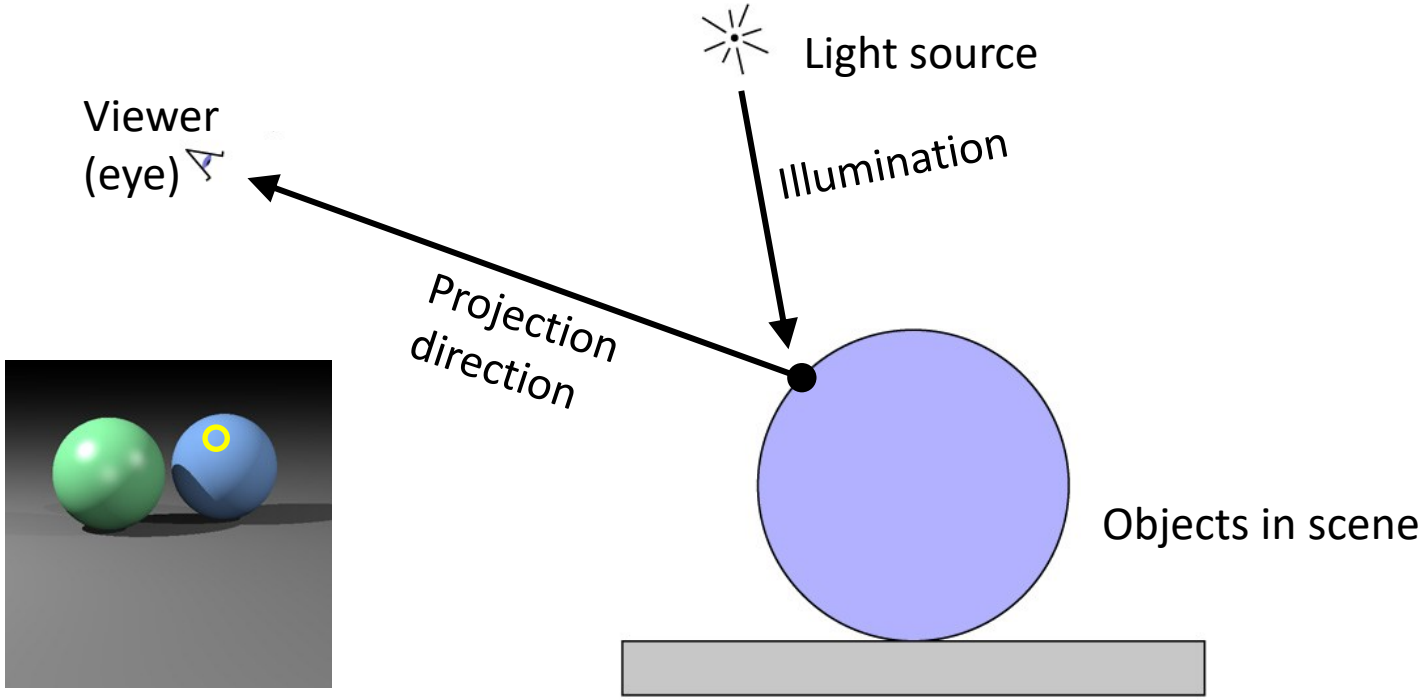
- Most basic way to shade a surface: take dot product of unit surface normal (N) and unit direction to light (L)

```
double surfaceColor( Vec3 N, Vec3 L ) {
        return max( 0, dot( N, L ) );
}
```

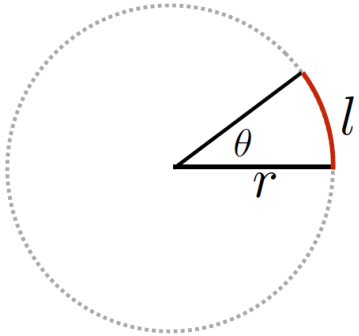


There is more to shading than N dot L

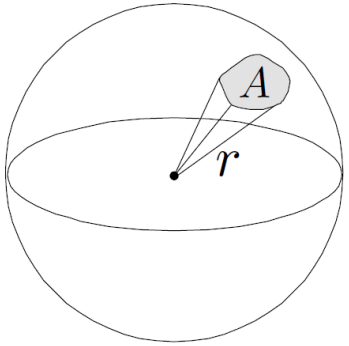
- Shading is the process computing color of a surface point in a 3D scene, based on things like the surface's angle to lights, its distance from lights, its angle to the camera and material properties



Angles and Solid Angles



- Angle: ratio of subtended arc length on circle to radius
  - $\theta = \frac{l}{r}$
  - Circle has 2π radians
- Solid angle: ratio of subtended area on sphere to radius squared
  - $\Omega = A/r^2$
  - Sphere has 4π steradians
- Differential solid angle $d\omega$
  - Consider a tiny area (differential area dA) projected onto the surface of the unit spere

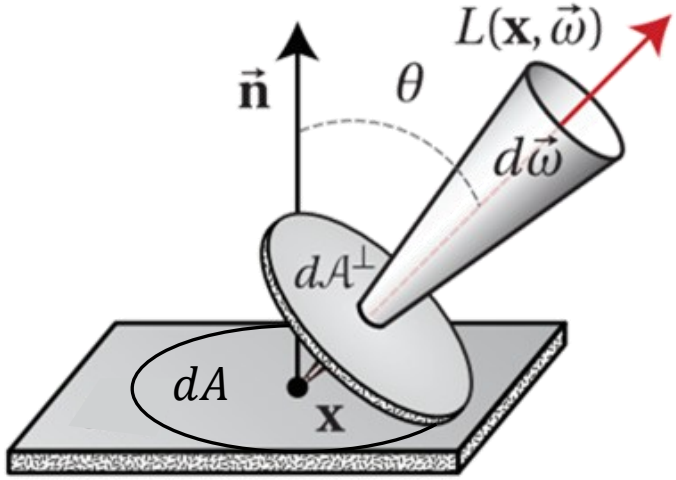


How much light is reflected, in what directions?

- Radiance is power (radiant flux, Φ) per unit area per solid angle
- Surface radiance is power per unit area per solid angle measured perpendicularly to a surface

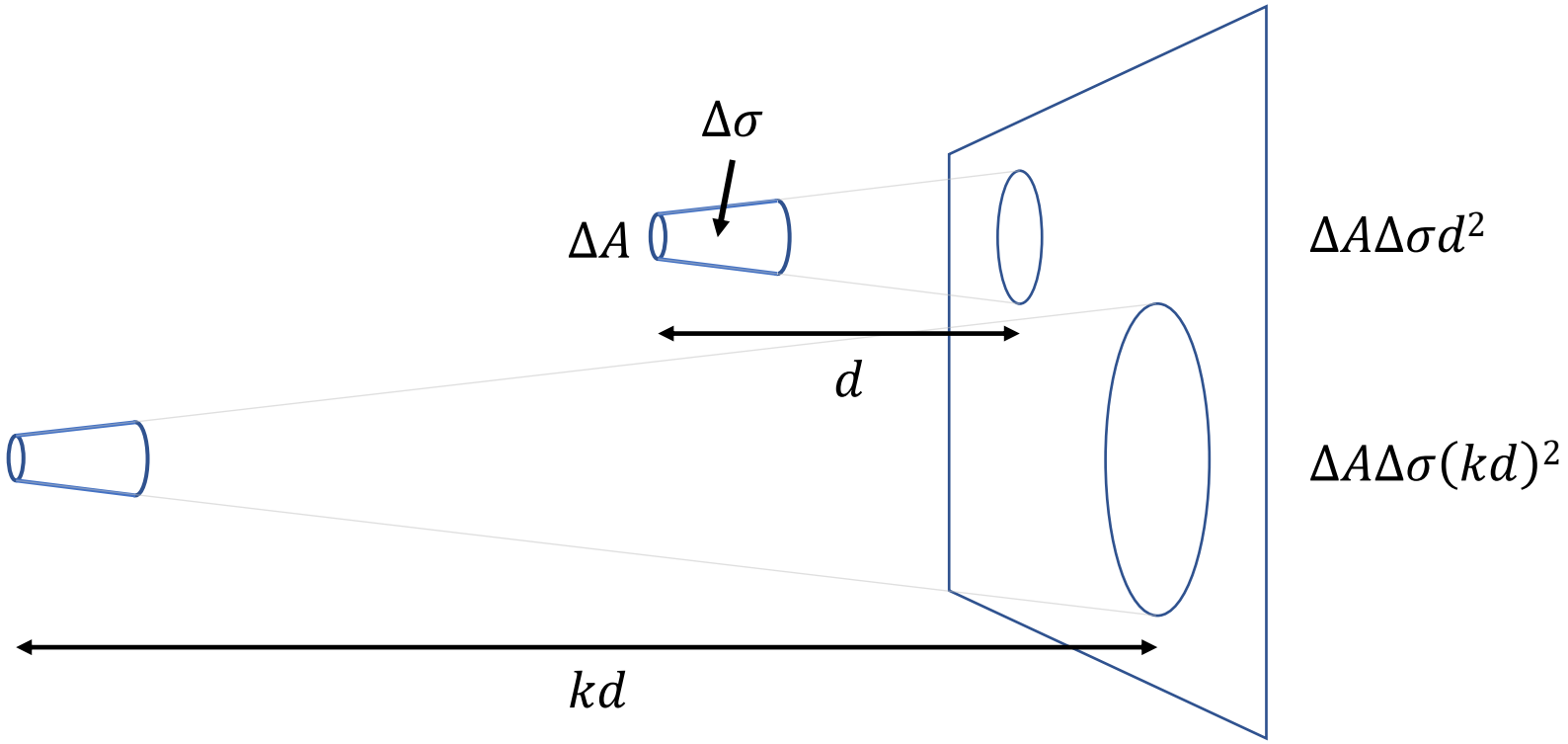
$$L = \frac{d^2\Phi}{dA^{\perp} d\vec{\omega}} = \frac{d^2\Phi}{dA\,(\vec{n}\,\cdot\vec{\omega})\,d\vec{\omega}}$$

- Note cosine of angle is ratio between $dA^{\perp}$ and dA
- ***The most important quantity in rendering!***
- This is the pixel "intensity" you observe



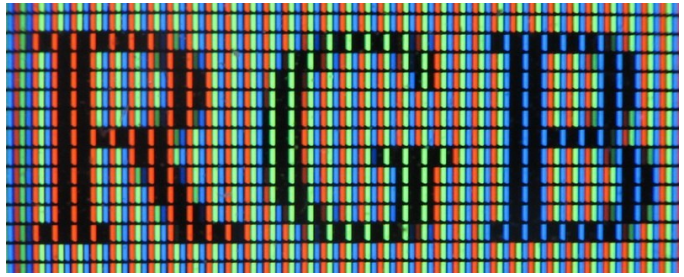
Radiance does not vary along a line in space

- Light inversely proportional to distance squared
- Area of surface sampled grows with distance squared



Spectral Radiance

- Radiance (recall):
  - Radiant energy per unit time per unit area per unit solid angle
- Spectral Radiance:
  - Radiant energy per unit time per unit area per unit solid angle per unit wavelength
- Needed to process information concerning colour
- We will discuss color perception later!
- For now, we know the simple solution is to work with 3 intensities to define a mix of red, green, and blue light

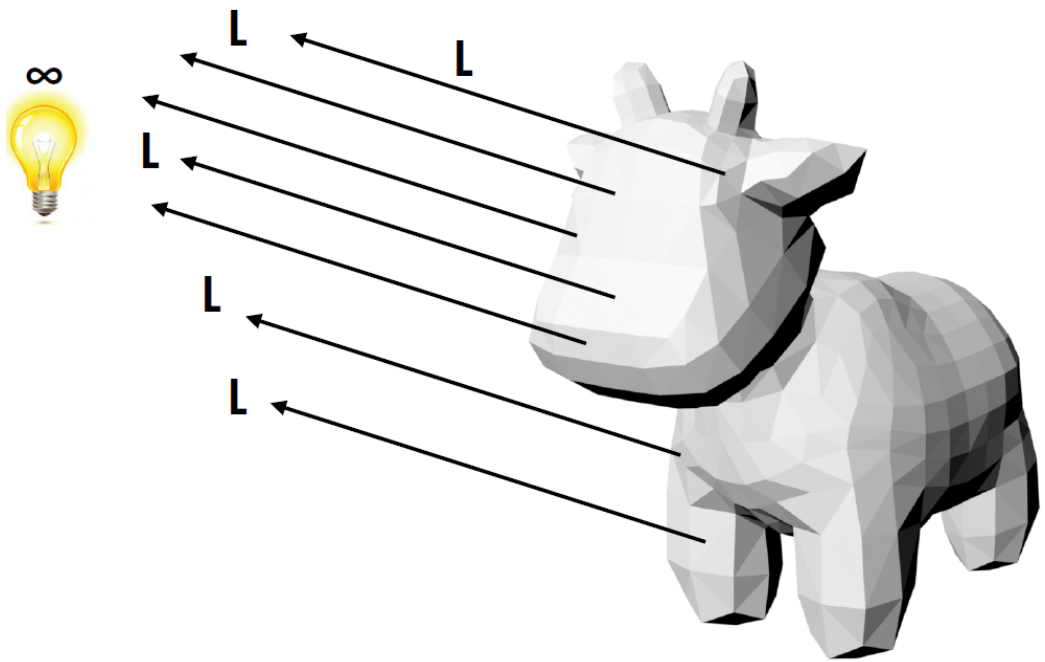


Incoming light and shading

- Should use radiance to define incoming light
  - Integrate over all incoming directions at a shading point
  - This leads to the rendering equation (more later)
- But consider simpler models first!
  - Directional lights
  - Point lights

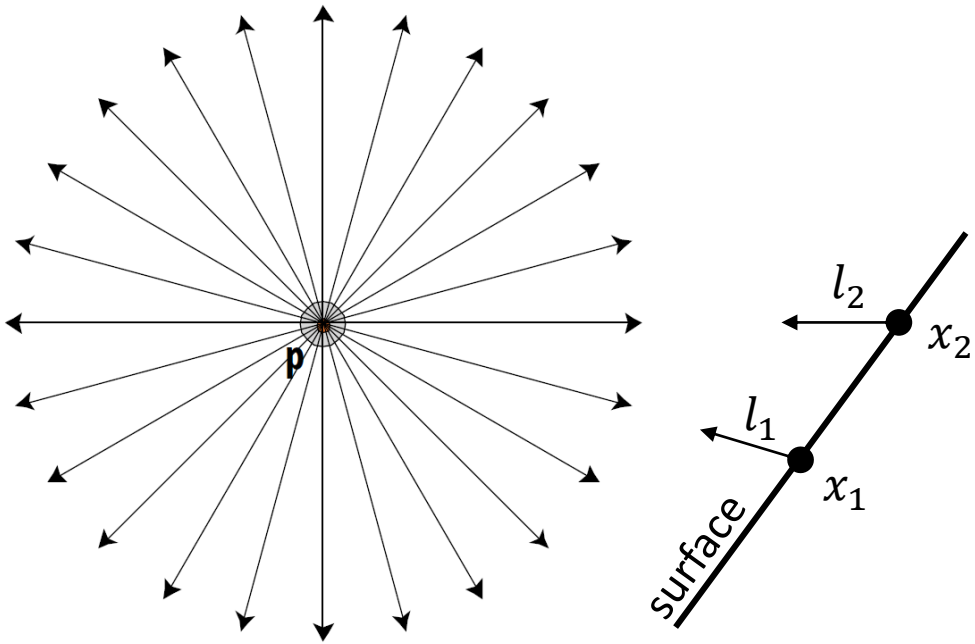
Directional Light

- Common abstraction
  - Infinitely distant light source providing illumination from a constant direction
  - Provides same power to all points in the scene (if not in shadow)



Isotropic point light

- Slightly more realistic
- Provides same energy in all directions (isotropic)
- Different surface points have different light directions



Total radiant flux

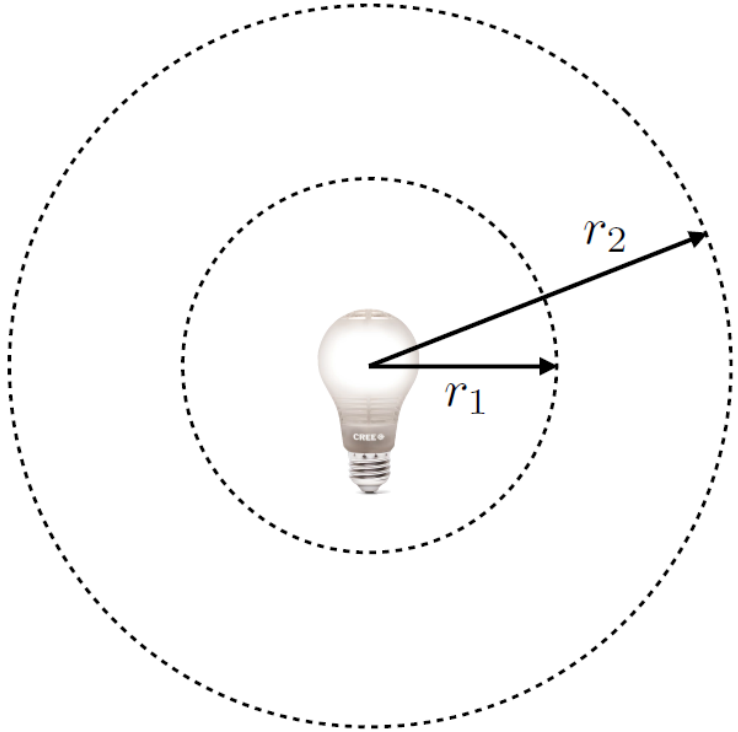
"Intensity", irradiance

$$\Phi = \int_{S^2} I \, d\omega$$
$$= 4\pi \, I$$

$$I = \frac{\Phi}{4\pi}$$

Irradiance fall off with distance

- Assume light is emitting flux Φ in a uniform angular distribution
- Same amount of power is distributed over larger and larger spheres
- Diminishes *quadratically* with distance
  - Can compare irradiance E at surface to two spheres



$$E_1 = \frac{\Phi}{4\pi r_1^2}$$

$$E_2 = \frac{\Phi}{4\pi r_2^2}$$

$$\frac{E_1}{E_2} = \frac{r_1^2}{r_2^2} = \left(\frac{r_1}{r_2}\right)^2$$

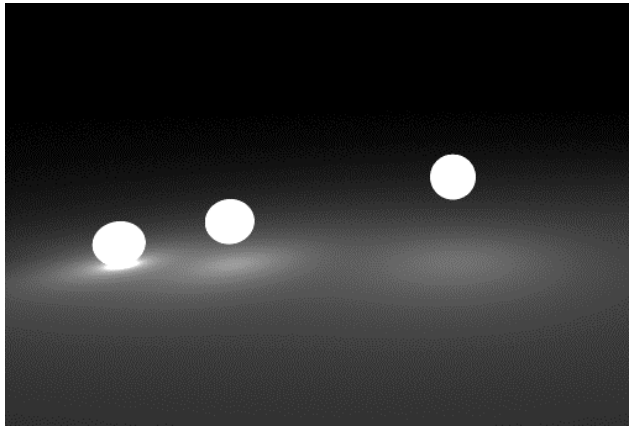
What happens as r_2 goes to zero?

Light attenuation approximations in practice

- Point lights are not realistic, and give bad results when too close to shading points
- Old OpenGL trick: attenuation factor as a quadratic function of distance d

$$\frac{1}{k_c + k_l d + k_q d^2}$$

- *Approximates* the behaviour of a small area light, but how to pick constants?
  - Tune by hand for desired aesthetic effect
  - Can model based on sphere radius r and distance to sphere
    - http://imdoingitwrong.wordpress.com/2011/01/31/light-attenuation/
  - Can model based on disc radius r and distance to disc
    - http://www.cemyuksel.com/research/pointlightattenuation/
  - See links for more information, but we will not cover the derivations in this course

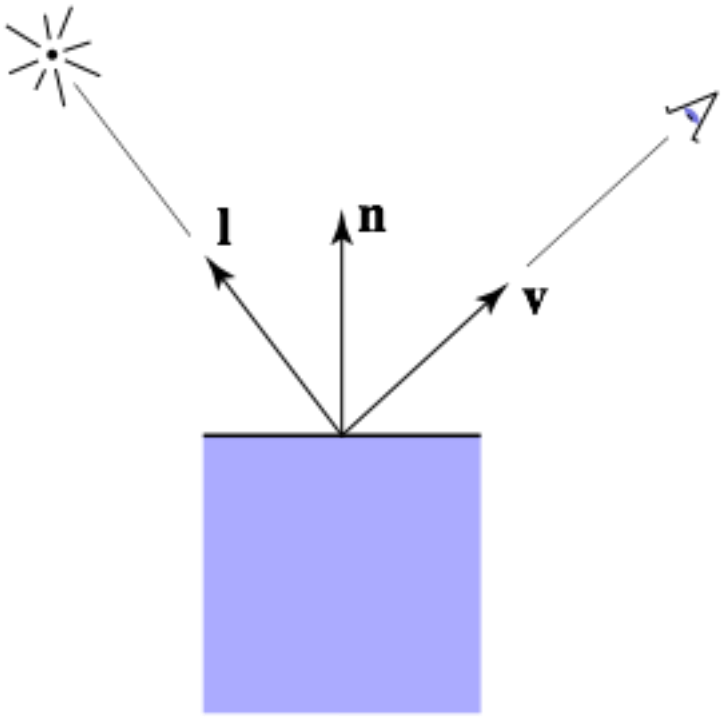


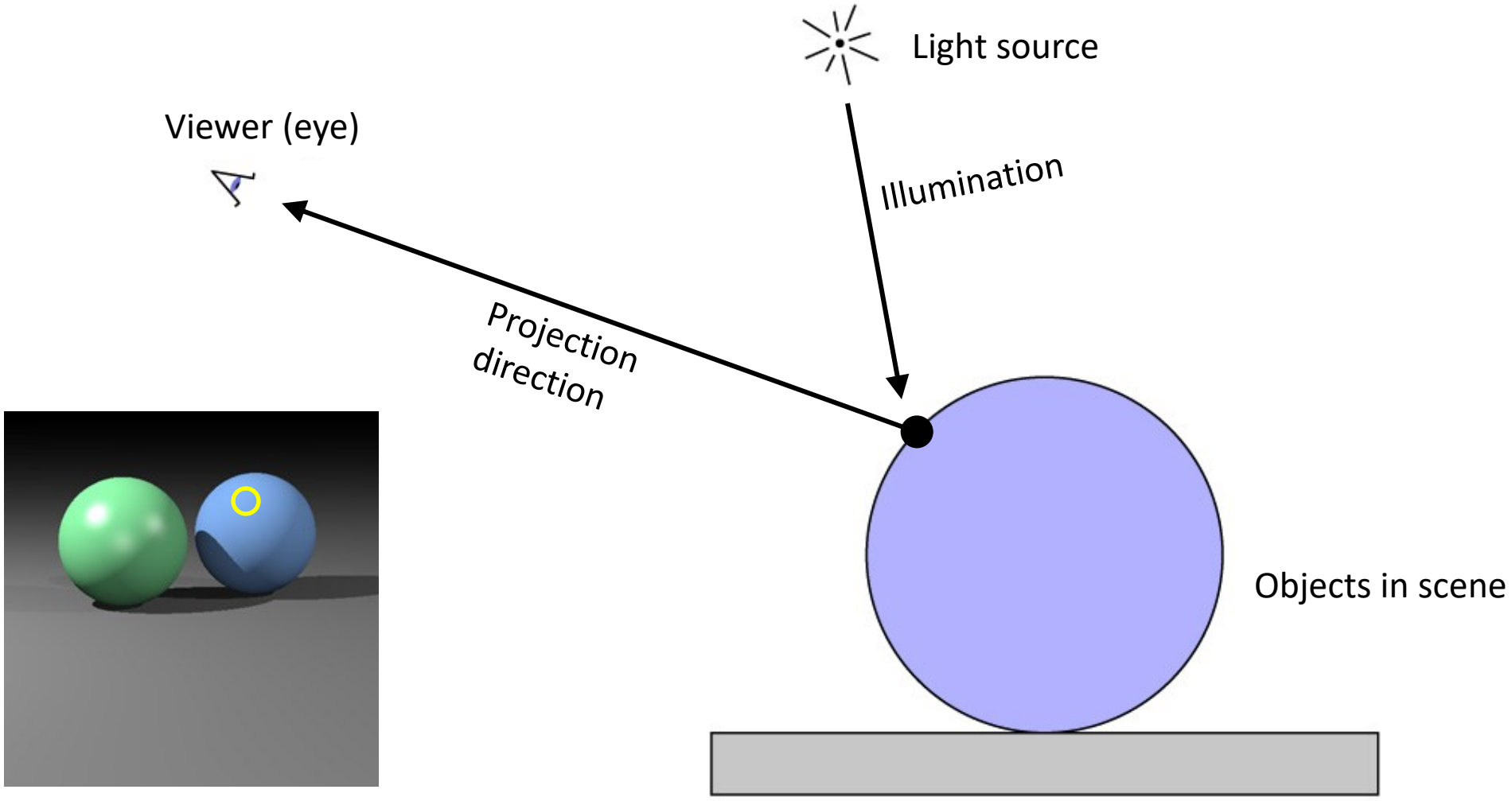
Light intensity in OpenGL

- Red, green, and blue intensities used to describe the colour of a light
- In OpenGL intensities, set typically set intensity $I \in [0,1]^3$
- Attenuation (if used) then diminishes these values
- Goal is to compute pixel color, $c = (r, g, b) \in [0,1]^3$
- Light intensities can be set larger than 1, but can cause saturated pixels (full brightness) if not reduced by attenuation or reflectance

Back to shading, how is light reflected?

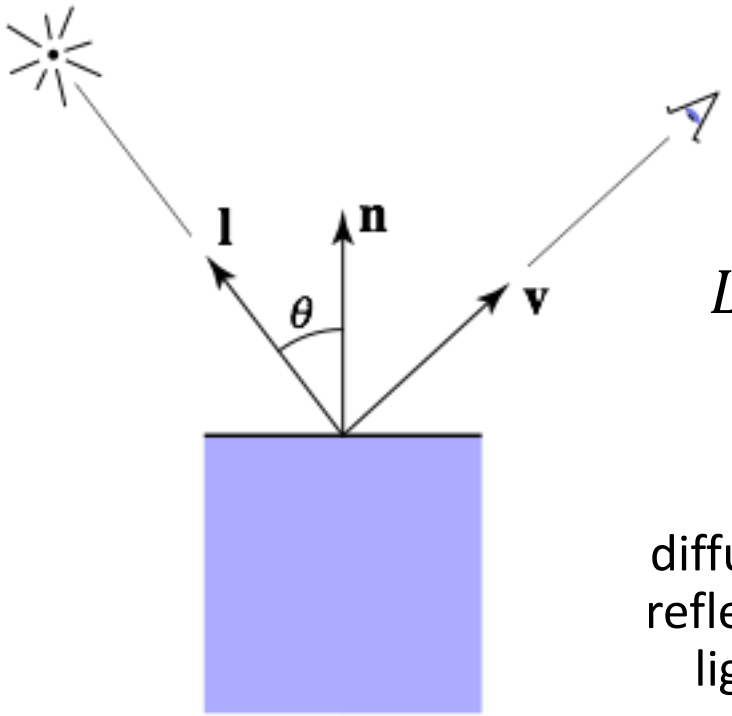
- Compute light reflected (*scattered*) toward camera
- Inputs:
  - eye direction
  - light direction
    (for each of many lights)
  - surface normal
  - surface parameters
    (color, shininess, …)




Light source
Viewer (eye)
Illumination
Projection direction
Objects in scene

Diffuse Lambertian shading

- A Lambertian surface scatters light equally in all directions
- Process each colour channel independently
- Let k_d and I be vectors with red greed and blue components, then see $k_d I$ as ***component wise multiplication***
- Coefficient k_d is the diffuse reflectance coefficient



illumination from source

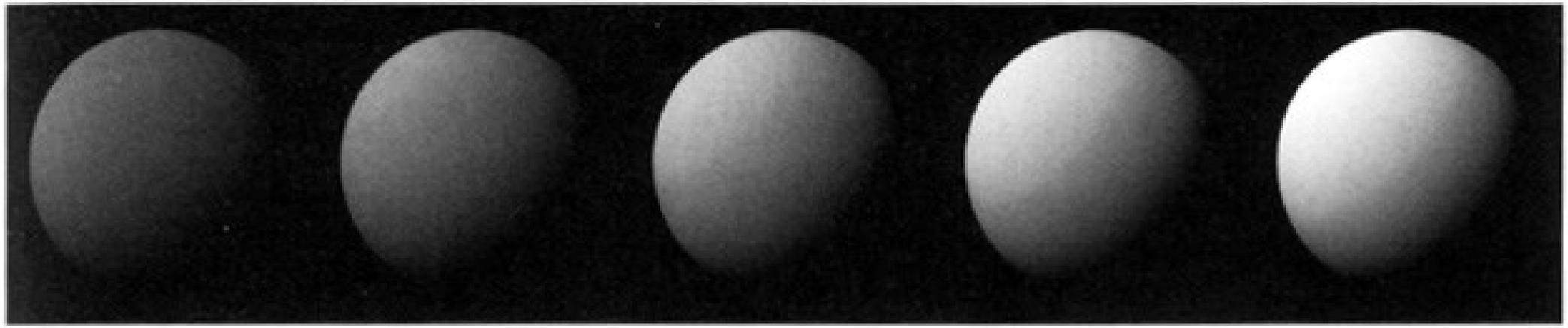
$$L_d = k_d \, I \, \max(0, \boldsymbol{n} \cdot \boldsymbol{l})$$

diffuse coefficient

diffusely reflected light

Lambertian shading

- Produces matte appearance



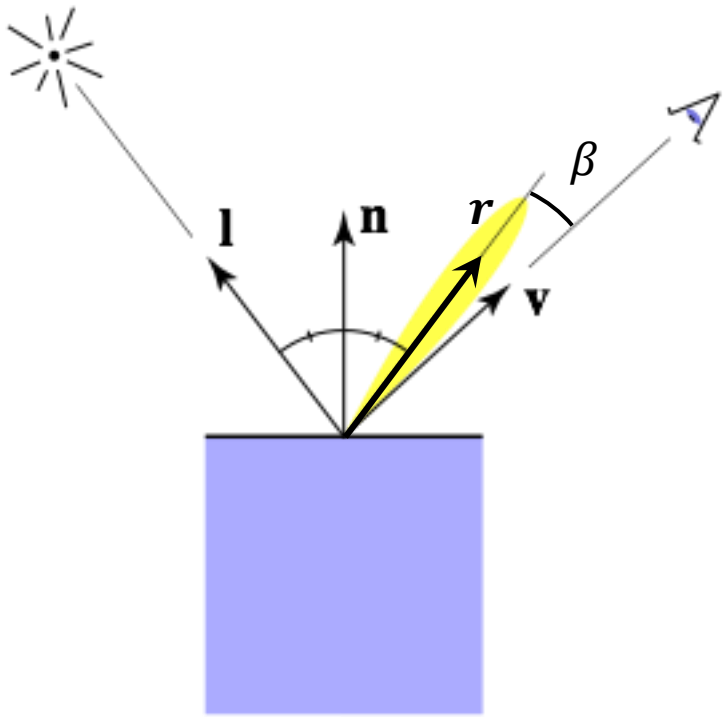
[Foley et al.]

k_d →

You may hear people talk of Albedo. Albedo is the fraction of *solar* light reflected by a surface.
Surface reflectance is the ratio of light not absorbed by a surface to light striking the surface.
Albedo is a measure of energy and Surface Reflectance is a property of a material. (effectively same thing in graphics)

Specular shading (Phong)

- Intensity depends on view direction
  - bright near mirror configuration



$$\boldsymbol{r} = \text{reflect}(\boldsymbol{l}, \boldsymbol{n}) = (2\boldsymbol{n}\boldsymbol{n}^T - \boldsymbol{I})\boldsymbol{l}$$

$$L_s = k_s\, I \max(0, \cos\beta)^p = k_s\, I \max(0, \boldsymbol{r} \cdot \boldsymbol{v})^p$$

specularly reflected light

specular coefficient

shininess

Phong model—plots

- Increasing specular exponent *p* narrows the lobe

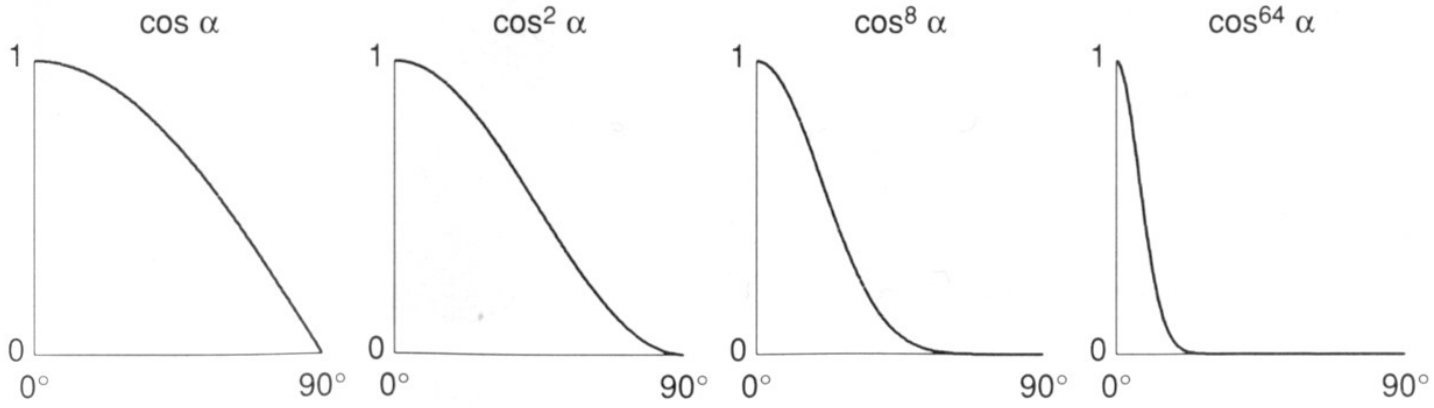


Fig. 16.9 Different values of $\cos^n \alpha$ used in the Phong illumination model.

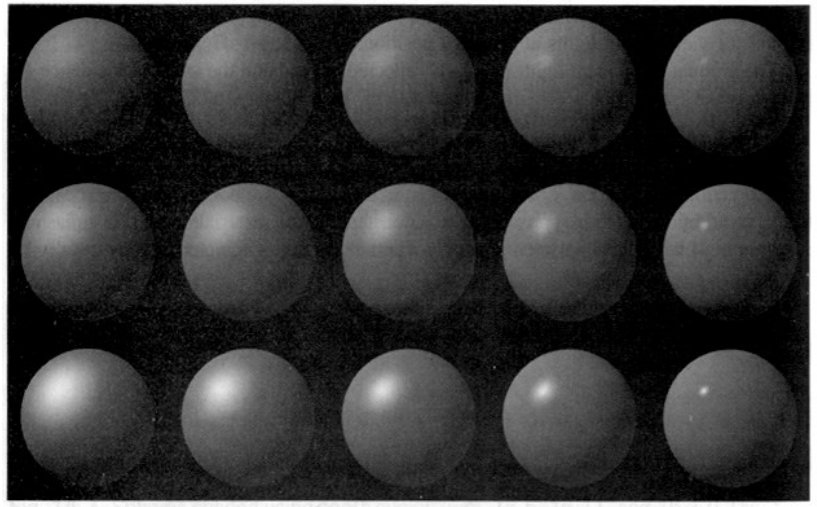
[Foley et al.]

- What are good values for the exponent?
- Alternatively, what are bad values? What can go wrong?
  - Note, the model is phenomenological… not based on physics

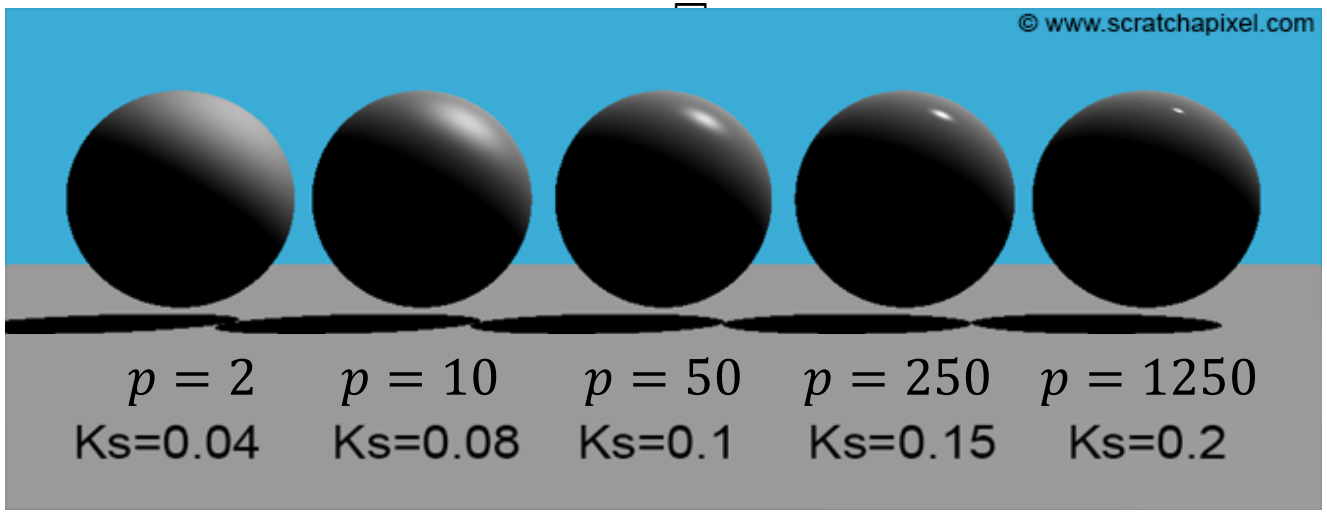
Specular shading

- When adjusting specular shading parameters, may want to change k_s while increasing the exponent to make plausible materials

k_s

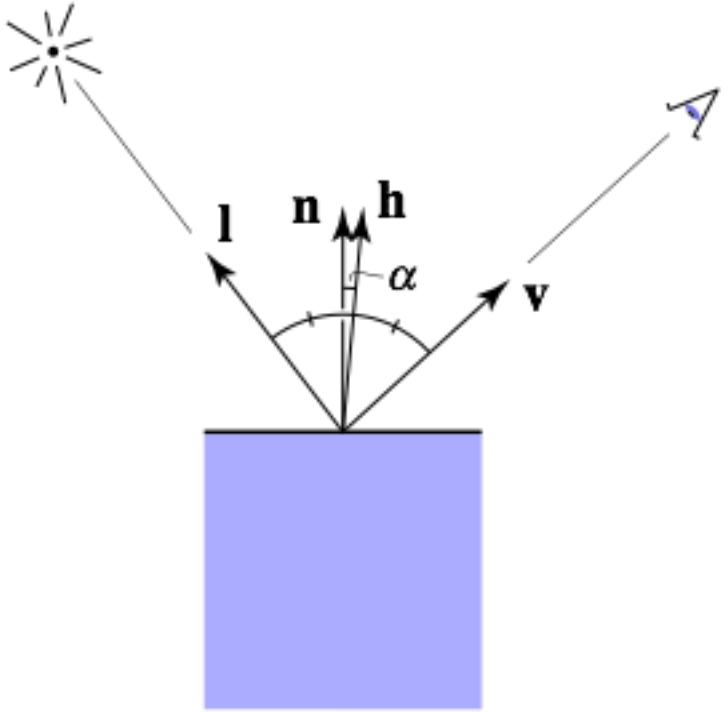


p



Specular shading (Blinn-Phong)

- Close to mirror ⇔ half vector near normal
  - Measure "near" by dot product of unit vectors



$$\boldsymbol{h} = \text{bisector}(\boldsymbol{v}, \boldsymbol{l}) = \frac{\boldsymbol{v} + \boldsymbol{l}}{\|\boldsymbol{v} + \boldsymbol{l}\|}$$

$$L_s = k_s \, I \, \max(0, \cos\alpha)^p = k_s \, I \, \max(0, \boldsymbol{n} \cdot \boldsymbol{h})^p$$

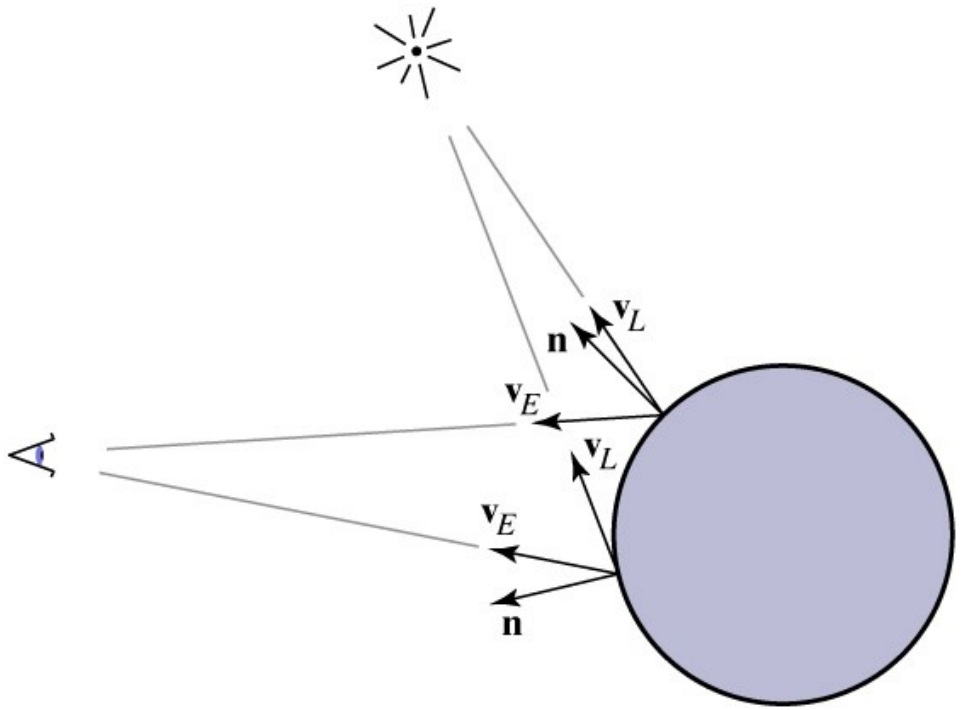
specularly reflected light

specular coefficient

shininess

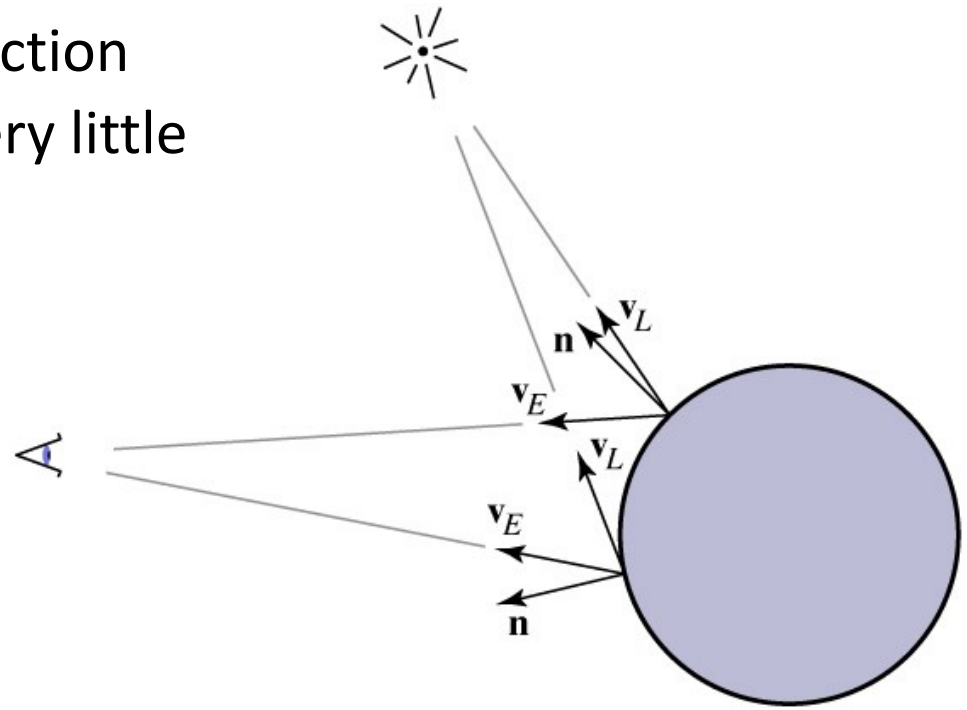
Local vs infinite viewer, point vs directional light

- Phong illumination requires geometric information:
  - light vector (function of position)
  - eye vector (function of position)
  - surface normal (from application)
- Light and eye vectors change
  - need to be computed (and normalized) for each shading point



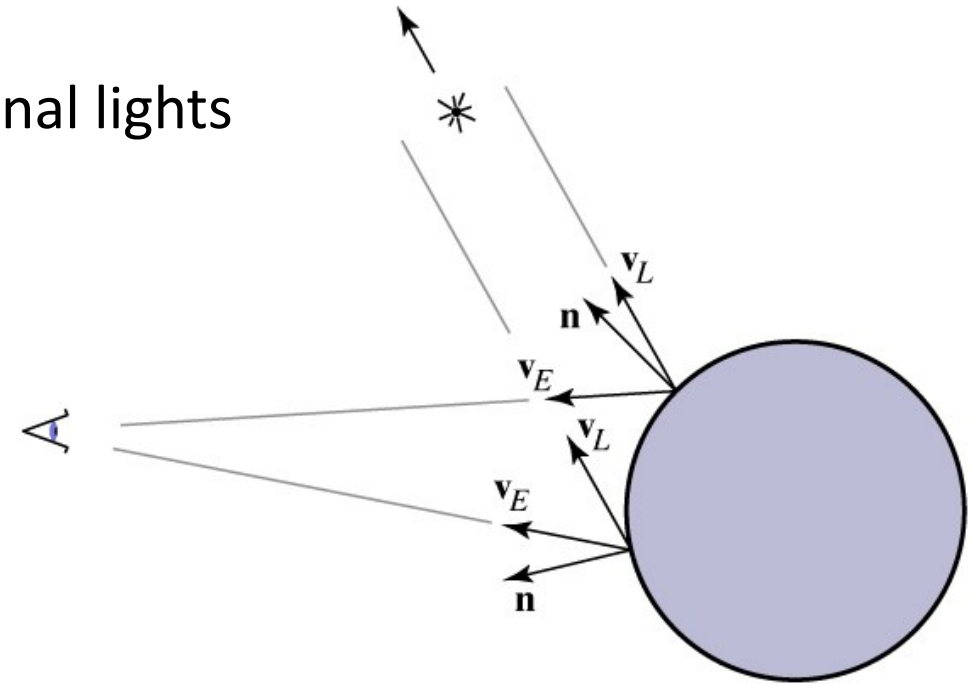
Local vs infinite viewer, point vs directional light

- Look at case when eye and light are far away:
  - Distant light source: nearly parallel illumination
  - Distant eye point: nearly orthographic projection
  - In both cases, eye or light vector changes very little
- Optimization:
  - Approximate eye as infinitely far away
  - Use directional light



Directional light

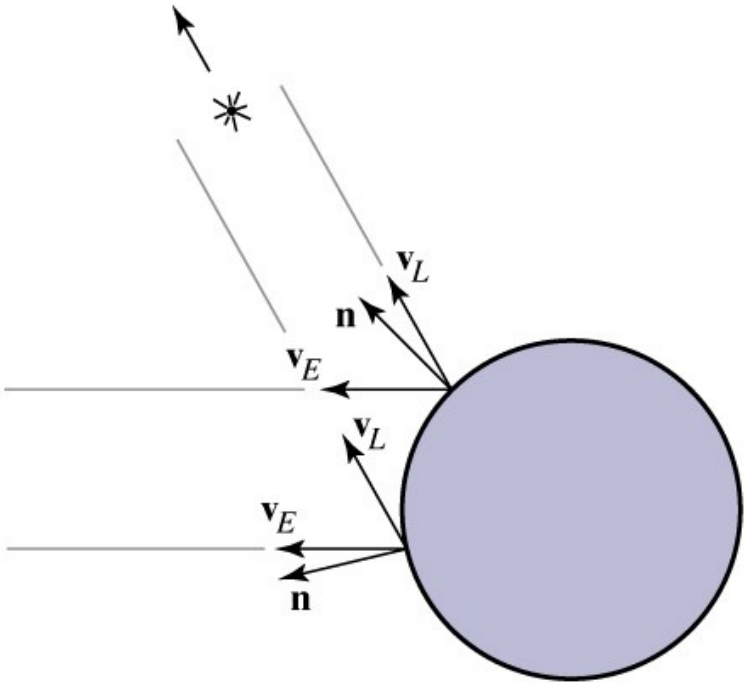
- Directional (infinitely distant) light source
  - light vector always points in the same direction
  - often specified by position [*x* *y* *z* 0]
  - many pipelines are faster if you use directional lights



Infinite viewer

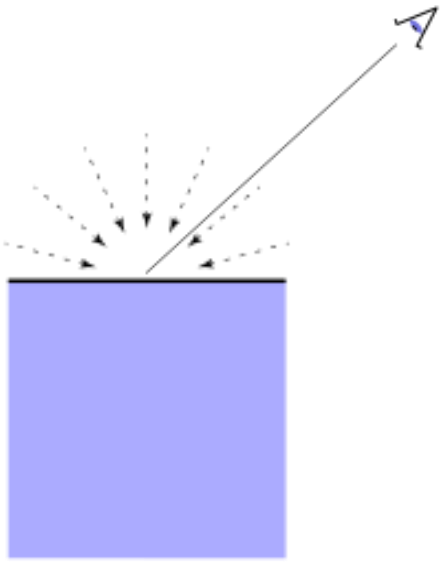
- Orthographic camera
  - projection direction is constant
- "Infinite viewer" approximation
  - With perspective, can approximate eye vector as z, i.e., using the image plane normal
    - However, can produce weirdness for wide-angle views
  - Blinn-Phong advantage over Phong
    - Light, eye, half vectors all constant!
    - Thus, constant half angle
    - Specular component only depends on normal

$$L_s = k_s \, I \, \max(0, \boldsymbol{n} \cdot \boldsymbol{h})^p$$



Ambient shading

- Shading that does not depend on anything
  - Add constant color to account for ***disregarded illumination***
  - This fills in black shadows
  - What is this "disregarded illumination" that we didn't include?



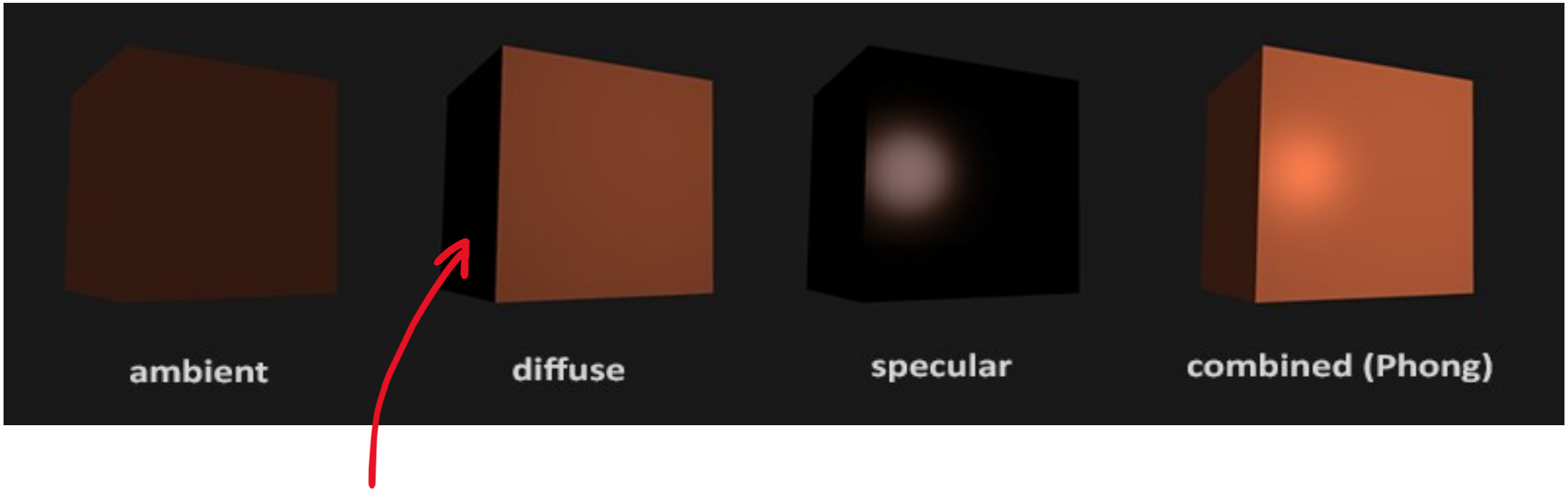
$$L_a = k_a \, I_a$$

reflected ambient light

ambient coefficient (typically same as the diffuse Coefficient)

ambient light intensity

Ambient shading, and combined shading



- Without ambient shading component, the left side of the cube would be invisible on a black background

Putting it together

- Usually include ambient, diffuse, Phong, all in one model

$$\begin{aligned} L &= L_a + L_d + L_s \\ &= k_a I_a + k_d\, I \max(0, \boldsymbol{n} \cdot \boldsymbol{l}) + k_s\, I \max(0, \boldsymbol{n} \cdot \boldsymbol{h})^p \end{aligned}$$

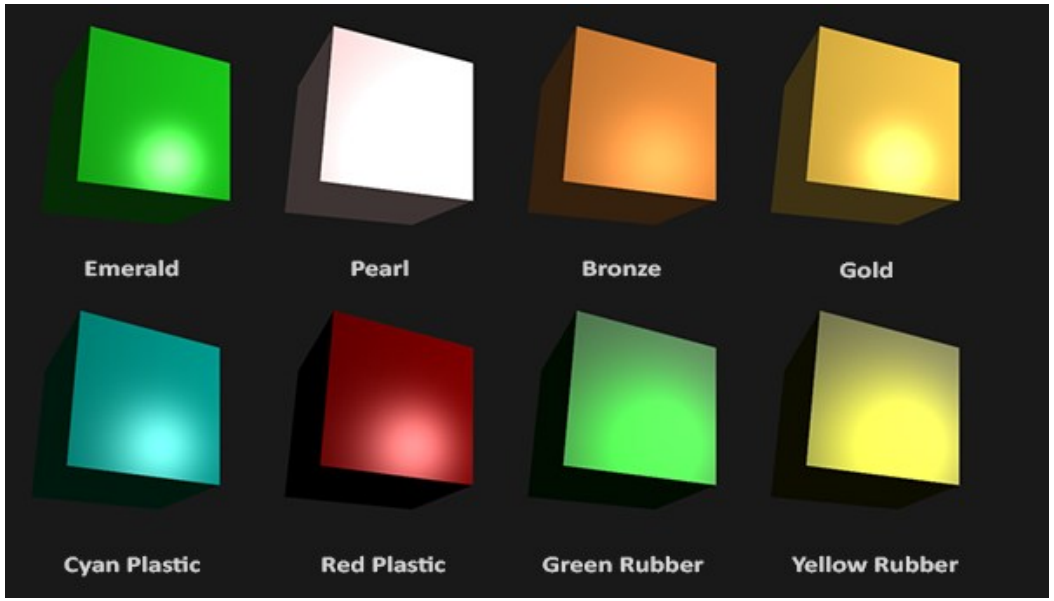
- The final result is the sum over many lights

$$\begin{aligned} L &= L_a + \sum_{i=1}^{N} \left((L_d)_i + (L_s)_i \right) \\ &= k_a I_a + \sum_{i=1}^{N} \left(k_d I_i \max(0, \boldsymbol{n} \cdot \boldsymbol{l}_i) + k_s I_i \max(0, \boldsymbol{n} \cdot \boldsymbol{h}_i)^p \right) \end{aligned}$$

- Easy to compute the final value using a GLSL vertex and fragment program!

Choosing Parameters

- Tune by hand for desired aesthetic effect
- Use values from old 1990s OpenGL demos
- For realistic materials and lighting must use physically based rendering and models instead



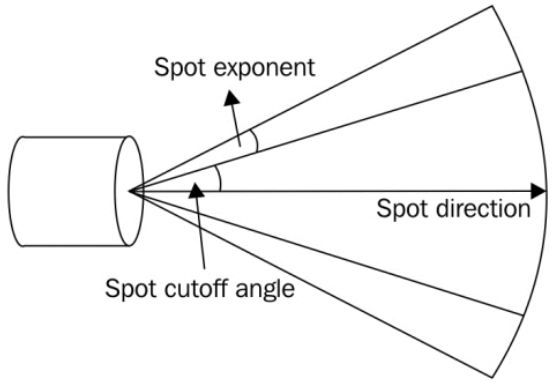
Name	Ambient			Diffuse			Specular			Shininess
emerald	0.0215	0.1745	0.0215	0.0756	0.61424	0.07568	0.633	0.727811	0.633	0.6
jade	0.135	0.2225	0.1575	0.54	0.89	0.63	0.316228	0.316228	0.316228	0.1
obsidian	0.0537	0.05	0.0662	0.1827	0.17	0.22525	0.332741	0.328634	0.346435	0.3
pearl	0.25	0.2072	0.2072	1	0.829	0.829	0.296648	0.296648	0.296648	0.088
ruby	0.1745	0.0117	0.0117	0.6142	0.04136	0.04136	0.727811	0.626959	0.626959	0.6
turquoise	0.1	0.1872	0.1745	0.396	0.74151	0.69102	0.297254	0.30829	0.306678	0.1
brass	0.3294	0.2235	0.0274	0.7803	0.568627	0.113725	0.992157	0.941176	0.807843	0.217948
bronze	0.2125	0.1275	0.054	0.714	0.4284	0.18144	0.393548	0.271906	0.166721	0.2
chrome	0.25	0.25	0.25	0.4	0.4	0.4	0.774597	0.774597	0.774597	0.6
copper	0.1912	0.0735	0.0225	0.7038	0.27048	0.0828	0.256777	0.137622	0.086014	0.1
gold	0.2472	0.1995	0.0745	0.7516	0.60648	0.22648	0.628281	0.555802	0.366065	0.4
silver	0.1922	0.1922	0.1922	0.5075	0.50754	0.50754	0.508273	0.508273	0.508273	0.4
black plastic	0.0	0.0	0.0	0.01	0.01	0.01	0.50	0.50	0.50	.25
cyan plastic	0.0	0.1	0.06	0.0	0.509803	0.509803	0.501960	0.501960	0.501960	.25
green plastic	0.0	0.0	0.0	0.1	0.35	0.1	0.45	0.55	0.45	.25
red plastic	0.0	0.0	0.0	0.5	0.0	0.0	0.7	0.6	0.6	.25
white plastic	0.0	0.0	0.0	0.55	0.55	0.55	0.70	0.70	0.70	.25
yellow plastic	0.0	0.0	0.0	0.5	0.5	0.0	0.60	0.60	0.50	.25
black rubber	0.02	0.02	0.02	0.01	0.01	0.01	0.4	0.4	0.4	.078125
cyan rubber	0.0	0.05	0.05	0.4	0.5	0.5	0.04	0.7	0.7	.078125
green rubber	0.0	0.05	0.0	0.4	0.5	0.4	0.04	0.7	0.04	.078125
red rubber	0.05	0.0	0.0	0.5	0.4	0.4	0.7	0.04	0.04	.078125
white rubber	0.05	0.05	0.05	0.5	0.5	0.5	0.7	0.7	0.7	.078125
yellow rubber	0.05	0.05	0.0	0.5	0.5	0.4	0.7	0.7	0.04	.078125

http://devernay.free.fr/cours/opengl/materials.html

Other effects

Can write GLSL code to compute other effects

- Spotlights are point lights with extra parameters
  - Position (same for point light)
  - Direction
  - Cutoff
  - Cutoff angle
  - Spot exponent
- See, e.g., OGLPG8
  - Classic Lighting Models in Chapter 7
  - Example 7.5



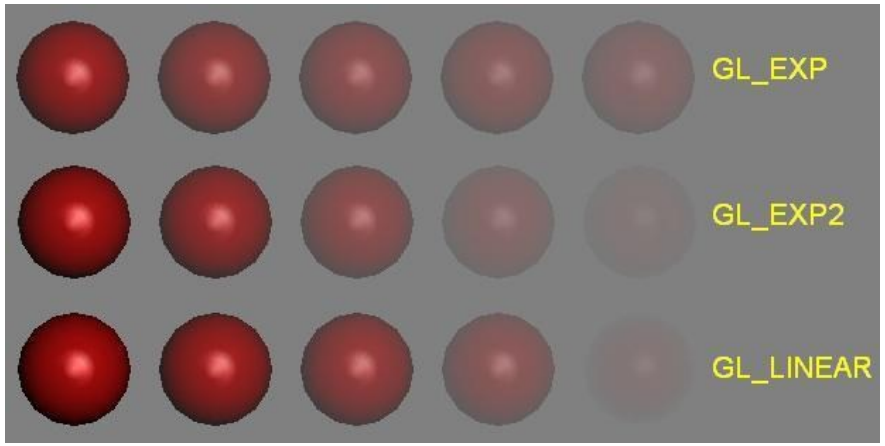
Light absorption and atmospheric scattering

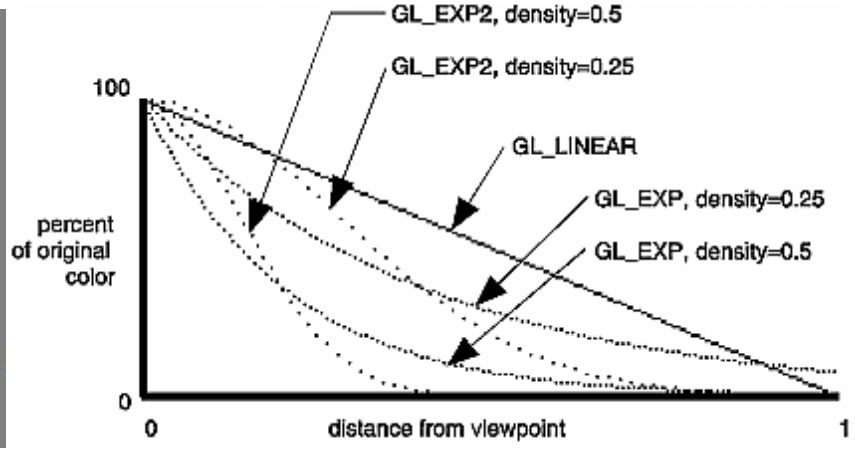
- Can mix the computed surface colour with a fog colour
  - Mix based on distance from viewer (i.e., z coordinate of point in eye coordinate frame)
  - Colour, exponential with density, or linear with start and end, using different equations from old-school OpenGL
  - Generally, a poor approximation (does not account for light positions)

$$f = \mathrm{e}^{-(density\ z)}$$

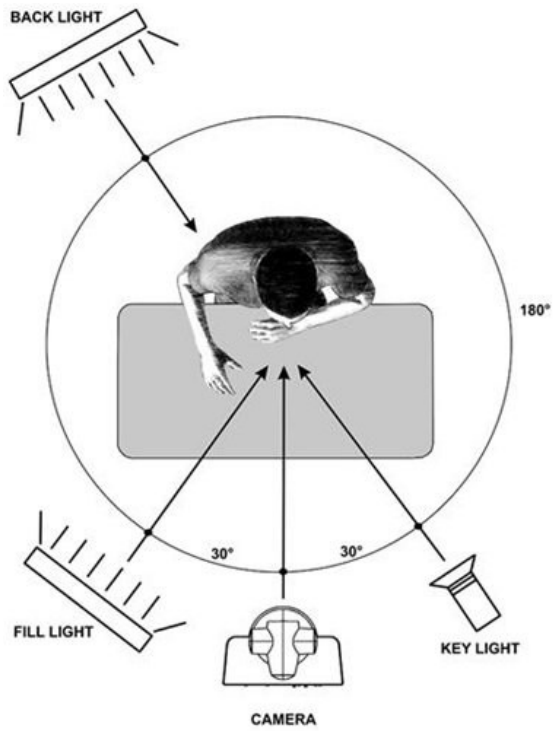
$$f = \mathrm{e}^{-(density\ z)^2}$$

$$f = \frac{end - z}{end - start}$$



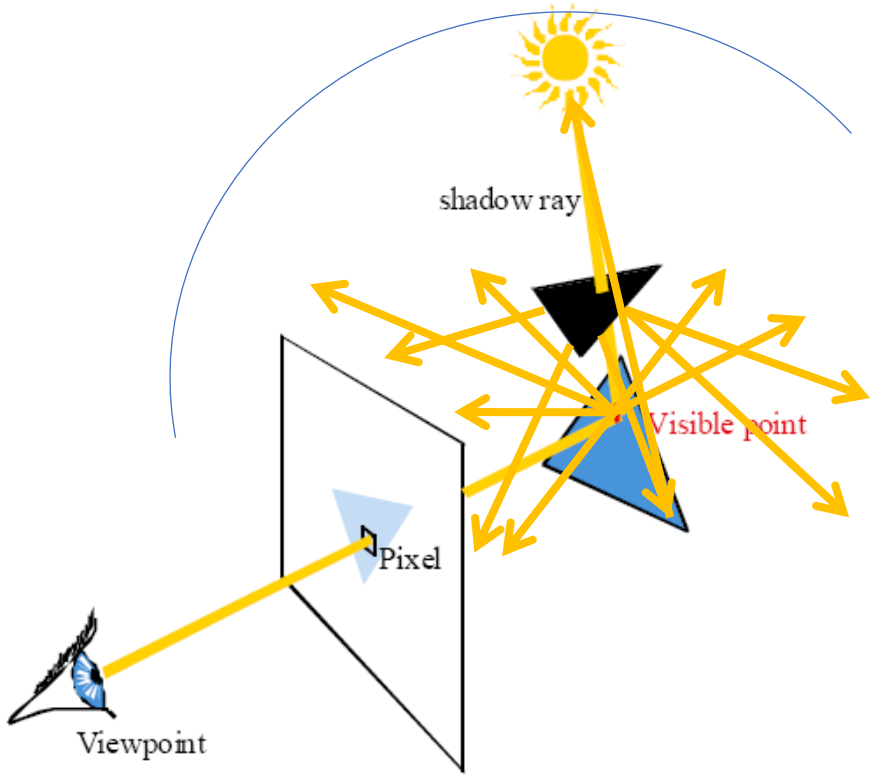


Lighting Design: How to make a scene look good?



- Completely black shadows are not so pleasing to the eye, so we add additional lights to fill in the shadows
- Aesthetically pleasing illumination can take inspiration from filmmaking!
- Three lights often using in television and film:
  - Key light, a bright light to reveal shape of actor
  - Fill light, reduces harsh shadows from key light
  - Back light, helps separate actor from background
- Often “area” lights are used, which is harder to reproduce in object order rendering, but can still go far with point lights!

Global Illumination – the harder problem

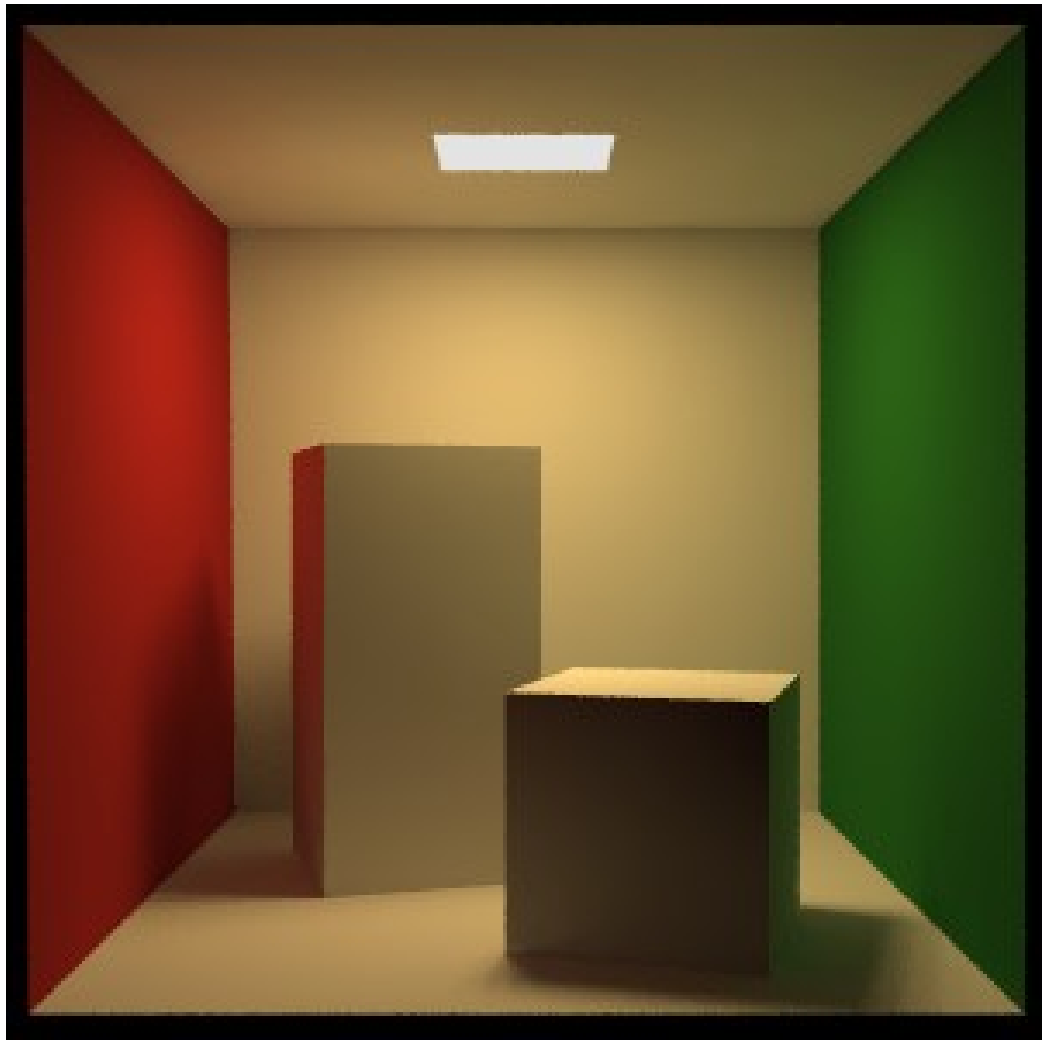


- Where does the light come from?
- Can still solve this problem for special cases, or with careful sampling in both object order and image order implementations

Global Illumination

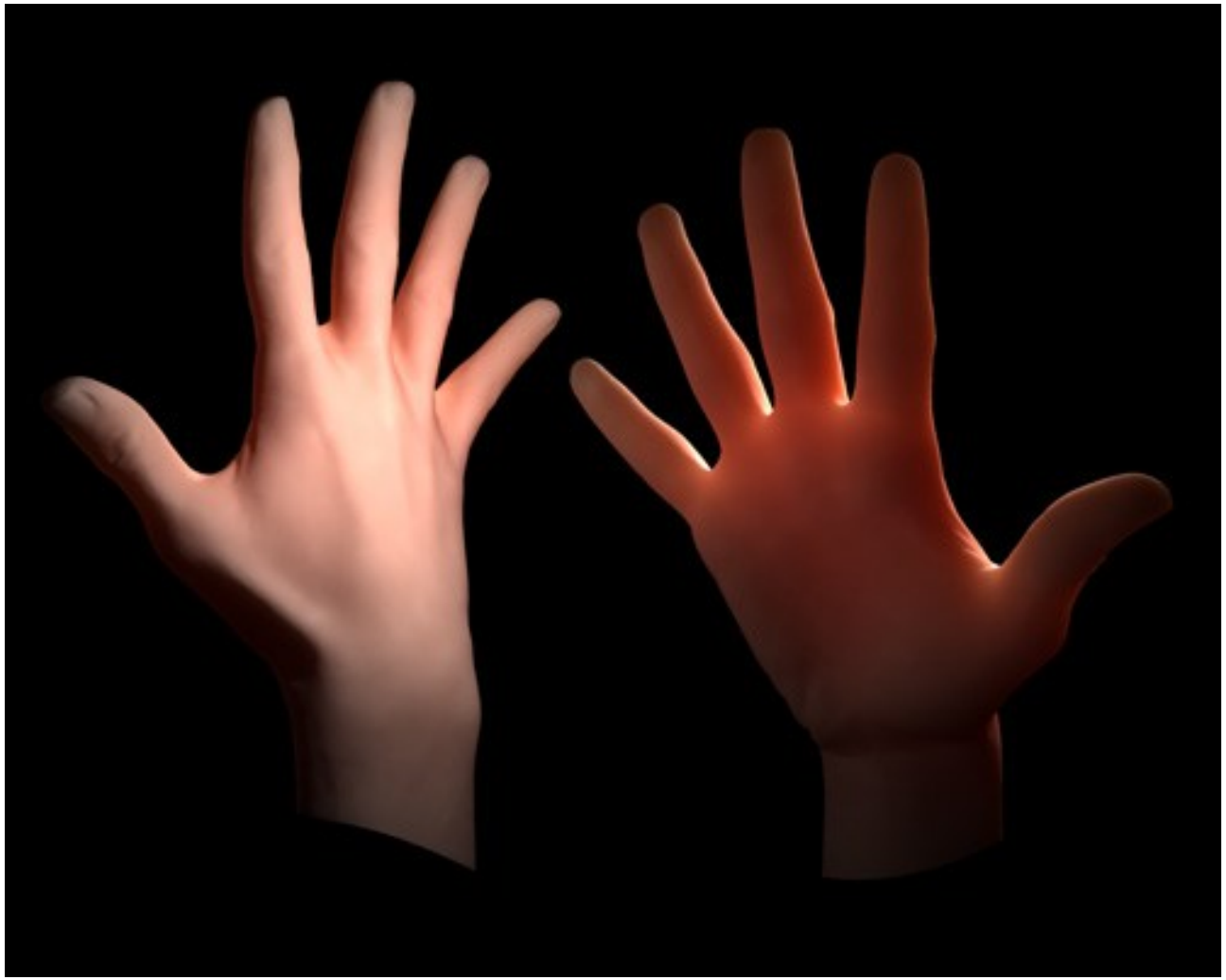
- Light can bounce off walls to illuminate different surfaces
  - Red reflected light illuminates left side of left object
  - Green reflected light illuminates right side of cube on the right

- We will revisit this problem later this term!



Lighting / Shading

- The full problem is even more complex!!!
  - Light is ***everywhere*** and can even be ***scattered beneath*** the surface of materials
  - Devising a global illumination method that is fast and that correctly models all ***light transport*** is the ultimate problem in rendering



[spin99, GELATO]

Review and More Information

- FCG Chapter 10, but not Section 10.3
  - Ambient, Diffuse, Specular (Phong)
  - Also, FCG Chapter 4 Section 4.5 on the basic shading equations in the context of ray tracing
  - Section 20.1 Radiometry